navi.

Agent Architecture &
Open-Source Integration Strategy

A practical blueprint for building a Claude Cowork agent team and integrating open-source ML/AI frameworks into the Navi development process.

Prepared for: Honeyheart (Matt)
Date: March 30, 2026
Version: 2.0

Contents

Section 1 Building a Claude Cowork Agent Team

- 1.1 Architecture Overview
- 1.2 The Four Agents: Chief, Researcher, Planner, Assistant
- 1.3 Step-by-Step Setup Guide
- 1.4 Administrative Tasks Catalog
- 1.5 Communication & Orchestration Patterns

Section 2 Open-Source Integration Strategy for Navi

- 2.1 Framework Evaluation & Decisions
- 2.2 The Navi Stack: What We Use, How, and Why
- 2.3 Decision Trees & Trade-offs
- 2.4 Phased Implementation Roadmap
- 2.5 Architecture for Nimble Expansion

Section 3 Wiring Open-Source Frameworks into Claude Cowork

- 3.1 Integration Methods: How Cowork Consumes External Tools
- 3.2 Framework-by-Framework Implementation Guide
- 3.3 File Structure & Where Everything Lives
- 3.4 Comparisons: Skills vs. MCP vs. Bash Scripts vs. Agent SDK
- 3.5 Practical Quickstart: Your First Cowork + Framework Session

SECTION 1: BUILDING A CLAUDE COWORK AGENT TEAM

1.1 Architecture Overview

A Claude Cowork agent team is a coordinated group of specialized Claude sessions, each with a distinct role, working together on shared tasks. The architecture mirrors a real-world executive team: a Chief who delegates and synthesizes, a Researcher who gathers intelligence, a Planner who designs strategy and timelines, and an Assistant who handles execution-level tasks. Each agent operates in its own context window with its own set of tools, skills, and MCP server connections.

This structure is achievable today using two mechanisms within the Claude ecosystem:

- **Subagents** (Claude Agent SDK) — A parent agent spawns child agents for focused subtasks. Children report results back to the parent. Best for hierarchical delegation where the Chief needs to maintain oversight.
- **Agent Teams** (Claude Code experimental) — Multiple Claude sessions with a shared task list, peer-to-peer messaging, and file locking. Best for parallel, collaborative work where agents need to coordinate directly.

> *Key insight: Start with subagents for simplicity. Graduate to Agent Teams only when agents genuinely need to talk to each other mid-task rather than just report results to the Chief.*

1.2 The Four Agents

Chief (CEO / Team Lead)

Role: Executive coordinator, decision-maker, quality gate

Goal: Decompose complex requests into subtasks, delegate to the right agent, synthesize outputs into a coherent deliverable, and make judgment calls.

Tools: TodoWrite, Agent (subagent spawner), all MCP connectors (Notion, Slack, etc.), file read/write, web search

Key Tasks:

- Receives user requests and breaks them into actionable subtasks
- Assigns tasks to Researcher, Planner, or Assistant based on task type
- Reviews and synthesizes outputs from all agents before presenting to user
- Makes prioritization and scope decisions when trade-offs arise
- Maintains the master task list and tracks progress across the team
- Escalates ambiguous or high-stakes decisions back to the user

- Manages the overall project timeline and deadlines

Researcher

Role: Intelligence gatherer, analyst, fact-checker

Goal: Find, validate, and synthesize information from the web, codebase, documentation, and connected tools.

Tools: WebSearch, WebFetch, Grep, Glob, Read, Notion search, Slack search, file analysis tools

Key Tasks:

- Conducts web research on technologies, competitors, market data
- Searches internal knowledge bases (Notion, Slack, files) for context
- Analyzes codebases to understand architecture, patterns, and dependencies
- Fact-checks claims and validates assumptions before they reach the Planner
- Produces structured research briefs with sources and confidence levels
- Monitors for relevant updates, news, or changes in the problem domain
- Compares tools, frameworks, and approaches with pros/cons analysis

---

Planner

Role: Strategist, architect, roadmap designer

Goal: Transform research and requirements into structured plans, timelines, and decision frameworks.

Tools: TodoWrite, Read, Write, Edit, Notion (create/update), calendar tools, file tools

Key Tasks:

- Designs implementation plans with phased milestones and dependencies
- Creates project timelines, sprint plans, and task breakdowns
- Architects technical solutions based on Researcher's findings
- Writes decision documents with options, trade-offs, and recommendations
- Builds templates, frameworks, and reusable planning artifacts
- Identifies risks, blockers, and contingency plans
- Maintains living documents that evolve as the project progresses

---

Assistant

Role: Executor, builder, operations handler

Goal: Handle all execution-level tasks: writing code, creating files, managing data, formatting outputs, running scripts.

Tools: Bash, Write, Edit, Read, file tools, code execution, package managers, all creation tools (docx, pdf, pptx, xlsx)

Key Tasks:

- Writes and edits code (Python, JS, Swift, HTML/CSS, shell scripts)
- Creates documents, presentations, spreadsheets, and PDFs
- Runs scripts, tests, builds, and deployment commands
- Formats and transforms data between formats (CSV, JSON, etc.)
- Manages git operations (commits, branches, merges)
- Handles file organization, renaming, and cleanup
- Executes repetitive or mechanical tasks the other agents delegate
- Produces final deliverables in the correct format for the user

---

1.3 Step-by-Step Setup Guide

Step 1: Define Your CLAUDE.md Instructions

- Create a root CLAUDE.md file that describes the team structure, each agent's role, and delegation rules.
- Include standing rules: which agent handles which type of request, escalation paths, and output formats.
- Add a 'DO NOT MODIFY' list for files/systems that should never be changed without explicit approval.
- Specify the communication protocol: how agents report back, what format summaries take.

Step 2: Configure Skills Per Agent

- Chief gets: high-level orchestration skills, Notion/Slack connectors, summary generation.
- Researcher gets: web search, codebase exploration, document analysis, knowledge retrieval.
- Planner gets: document creation (docx, md), timeline builders, architecture diagramming.
- Assistant gets: code execution, file creation (pdf, pptx, xlsx, docx), git operations, shell access.
- Use the skills/ directory to assign specialized skill files to each agent's context.

Step 3: Wire Up MCP Servers

- Connect Notion for project documentation, task tracking, and knowledge base access.
- Connect Slack (if used) for team communication monitoring and notifications.
- Connect any domain-specific tools (analytics, CRM, etc.) via MCP registry.
- Each agent should only have the MCP connections relevant to its role (principle of least privilege).

Step 4: Establish the Delegation Protocol

- User request arrives at Chief. Chief analyzes and decides: research needed? planning needed? or straight execution?
- If research is needed: Chief spawns Researcher subagent with a specific brief.
- If planning is needed: Chief sends Researcher's output to Planner with constraints and goals.
- For execution: Chief assigns the plan to Assistant with clear acceptance criteria.
- Chief reviews all outputs before presenting to the user. Nothing goes to the user unreviewed.

Step 5: Set Up Memory and Continuity

- Use the auto-memory system to persist learnings across sessions (user preferences, project context, feedback).
- Create memory files for: user profile, project state, feedback/corrections, external references.
- Each agent should read relevant memories at session start to maintain continuity.
- Chief is responsible for writing new memories when important decisions or learnings emerge.

Step 6: Test with a Real Workflow

- Start with a concrete task: 'Research the best Telegram bot hosting options and create a comparison doc.'
- Chief decomposes: Research (Researcher) → Compare & Recommend (Planner) → Create Doc (Assistant).
- Observe where handoffs break down, where context is lost, where quality drops.
- Iterate on the CLAUDE.md instructions and agent configurations based on what you learn.

1.4 Complete Administrative Tasks Catalog

Below is a comprehensive list of administrative tasks the agent team can handle, organized by domain. Each task is tagged with the primary agent responsible.

Project Management

Task	Agent
Create and maintain project timelines	Planner
Break epics into tasks and subtasks	Planner
Track task completion and progress	Chief
Generate status reports and summaries	Chief
Identify blockers and propose mitigations	Planner
Manage sprint planning and retrospectives	Planner
Maintain changelog and release notes	Assistant

Research & Analysis

Task	Agent
Competitive analysis and market research	Researcher
Technology evaluation and comparison	Researcher
User feedback synthesis and pattern identification	Researcher
Pricing research and cost modeling	Researcher
Regulatory/compliance research	Researcher
API documentation review and integration feasibility	Researcher
Performance benchmarking and analysis	Researcher

Content & Documentation

Task	Agent
Write and update technical documentation	Assistant
Create presentations (pitch decks, reports)	Assistant
Draft blog posts, newsletters, marketing copy	Assistant
Generate PDF reports and spreadsheets	Assistant

Create onboarding guides and SOPs	Planner
Write meeting agendas and minutes	Planner
Maintain README files and API docs	Assistant

Code & Development

Task	Agent
Write new features and modules	Assistant
Debug and fix issues	Assistant
Code review and quality analysis	Researcher
Refactoring and optimization	Assistant
Write tests and validation scripts	Assistant
Git operations (branch, commit, merge, PR)	Assistant
Dependency management and updates	Assistant
Architecture design and system diagramming	Planner

Operations & DevOps

Task	Agent
Server provisioning and configuration	Assistant
Deployment scripts and CI/CD setup	Assistant
Log analysis and error triage	Researcher
Performance monitoring and alerting setup	Assistant
Database schema design and migration	Planner
Backup and recovery planning	Planner

Communication & Outreach

Task	Agent
Draft emails, messages, and announcements	Assistant
Social media content planning	Planner
Conference/event preparation materials	Planner
Customer communication templates	Assistant

Investor update drafts	Planner
Partnership outreach research	Researcher

Data & Analytics

Task	Agent
Data cleaning and transformation	Assistant
Dashboard creation and visualization	Assistant
Survey design and analysis	Planner
Metrics definition and tracking setup	Planner
A/B test design and result analysis	Researcher
User behavior pattern analysis	Researcher

Financial & Legal

Task	Agent
Budget tracking and expense categorization	Assistant
Invoice and receipt organization	Assistant
Contract clause research (not legal advice)	Researcher
Financial projection modeling	Planner
Tax deadline and compliance tracking	Planner

1.5 Communication & Orchestration Patterns

Sequential Pipeline (Most Common)

User → Chief → Researcher → Planner → Assistant → Chief → User. This is the default flow for most complex tasks. Each agent adds value in sequence, and the Chief acts as quality gate at both entry and exit.

Parallel Fan-Out

Chief spawns multiple agents simultaneously for independent subtasks. Example: Researcher investigates three competing frameworks while Assistant sets up the project scaffold. Results converge back at Chief for synthesis. This pattern cuts wall-clock time by 3-5x but costs proportionally more tokens.

Iterative Refinement Loop

Chief sends work to an agent, reviews the output, and sends it back with feedback for improvement. This is especially useful for creative work (copy, presentations, designs) where the first draft rarely hits the mark. Set a max iteration count (typically 2-3) to prevent infinite loops.

Direct Delegation (Simple Tasks)

For straightforward tasks, Chief skips the pipeline and goes directly to the responsible agent. 'Create a PDF of this data' goes straight to Assistant. 'What's the latest on X?' goes straight to Researcher. No overhead, no unnecessary steps.

> *Cost awareness: Agent Teams use roughly 7x the tokens of a single session. Use the simplest pattern that gets the job done. A solo Chief with occasional subagent calls covers 80% of daily work.*

SECTION 2: OPEN-SOURCE INTEGRATION STRATEGY FOR NAVI

2.1 Framework Evaluation & Decisions

The pasted material lists 12 open-source tools. Not all of them belong in Navi's stack. Below is a decisive evaluation of each, with a clear verdict: USE, DEFER, or SKIP. The filter: does this tool help Navi ship the conference test flight, scale the agent platform post-conference, or directly improve agent quality?

Framework	Verdict	Rationale
LangGraph	USE (Phase 2)	Best-in-class for stateful, multi-step agent workflows. Navi agents need memory, branching logic, and tool orchestration. This becomes the backbone of Navi's agent runtime after the conference MVP ships.
CrewAI	USE (Phase 1)	Fastest path to a working multi-agent prototype. Role-based setup maps directly to Navi's archetype model. Use for rapid iteration during and immediately after the conference.
LangChain	USE (Phase 2)	Needed for tool integrations, RAG pipeline, and retrieval. Don't use as the orchestration layer (LangGraph handles that), but use its ecosystem for connectors and chains.
AutoGen	DEFER	Powerful for agent-to-agent debate and critique, but adds complexity Navi doesn't need yet. Revisit when Navi has multiple specialized agents that need to negotiate or review each other's work.
Dify	USE (Phase 2)	Visual workflow builder for non-engineers. Valuable once Navi needs to let users customize their agent behavior. Also useful for internal rapid prototyping of new agent workflows.
Open WebUI	SKIP	Self-hosted chat UI. Navi already has its own interface (Telegram now, native app later). No value-add.
Ollama	DEFER	Local model inference. Useful for cost reduction and offline testing, but Navi's current OpenRouter setup is sufficient. Revisit when inference costs become a real concern.
RAGFlow	DEFER	Retrieval-heavy app builder. LangChain's RAG primitives + a vector DB cover Navi's needs. Only revisit if retrieval becomes a core differentiator requiring specialized tooling.
PyTorch	DEFER	Core ML framework. Only needed if Navi trains custom models. Current architecture uses API-based LLMs. Revisit if/when Navi needs fine-tuned models for specific archetypes.
HuggingFace Transformers	DEFER	Same reasoning as PyTorch. Valuable for fine-tuning and custom inference, but premature before product-market fit is validated.

Optuna	SKIP	Hyperparameter optimization. Only relevant for custom model training, which is not on Navi's roadmap for 2026.
MLflow	DEFER	Experiment tracking. Useful once Navi is running systematic A/B tests on agent behavior. Too early now.

2.2 The Navi Stack: What We Use, How, and Why

The Conference Stack (Now → April 10)

The conference test flight is 11 days away. The bot is built. The stack stays simple: Python + python-telegram-bot + OpenRouter + SQLite. No framework additions before the conference. Every minute spent integrating a new framework is a minute not spent on the three things that actually matter: the Typeform handoff, the landing page, and QR cards.

> *Decision: Zero new frameworks before April 10. Ship the conference with what's built. Gather data. Then build the real platform.*

The Post-Conference Platform Stack

After the conference validates which archetype to build first, Navi transitions from a Telegram bot to a real agent platform. Here is the decisive stack, with clear roles for each component:

CrewAI — Agent Team Orchestration

Why: Maps directly to Navi's model. Each Navi archetype (barber assistant, attorney intake, etc.) is a 'crew' of 2-4 specialized agents working together. CrewAI's role/goal/backstory abstraction is a natural fit for Navi's personality-rich agents.

How: Each archetype defines its agents in YAML. The 'intake agent' handles first contact, the 'task agent' manages scheduling/reminders, the 'knowledge agent' handles domain-specific questions. CrewAI orchestrates the handoffs.

LangGraph — Stateful Workflow Engine

Why: Navi agents need to remember context across conversations, handle branching logic (if user needs X, do Y; otherwise do Z), and checkpoint state for recovery. LangGraph provides durable execution, human-in-the-loop oversight, and persistent memory.

How: Each agent's internal behavior is a LangGraph state machine. Nodes are actions (call LLM, use tool, wait for input). Edges are transitions. State is checkpointed after every step, so conversations can resume after failures or server restarts.

LangChain — Tool & Retrieval Layer

Why: Navi agents need to interact with external services (calendars, payment processors, booking systems) and retrieve knowledge from user-specific documents. LangChain's tool ecosystem and RAG primitives provide this without building everything from scratch.

How: Use LangChain's tool abstraction to wrap external APIs. Use its document loaders and retrievers to build per-user knowledge bases. Don't use LangChain for orchestration (that's LangGraph and CrewAI).

Dify — Visual Workflow Builder (Internal + User-Facing)

Why: Two use cases. Internally, Dify lets Matt prototype new agent workflows visually without writing orchestration code. User-facing, it could eventually let advanced users customize their agent's behavior through a visual interface.

How: Run Dify self-hosted alongside the main platform. Use it for internal prototyping first. Evaluate for user-facing customization only after the core agent experience is solid.

2.3 Decision Trees & Trade-offs

Decision 1: CrewAI vs. AutoGen for Multi-Agent

Factor	CrewAI	AutoGen
Setup complexity	Low — YAML config, CLI scaffold	Medium — async event-driven, more boilerplate
Agent definition	Role + Goal + Backstory (natural fit for Navi archetypes)	AssistantAgent / UserProxyAgent (more generic)
Learning curve for Matt	Gentle — vibecoder-friendly	Steeper — requires async/event patterns
Multi-agent collaboration	Sequential + hierarchical crews	Conversational + debate patterns
Production readiness	Good — active dev, commercial backing	Good — Microsoft-backed, enterprise focus
Best for Navi when?	Now through first 3 archetypes	Later, if agents need debate/critique loops

> *Verdict: CrewAI first. It's faster to stand up, friendlier for a vibecoder, and maps directly to Navi's archetype model. AutoGen is the upgrade path when agent-to-agent negotiation becomes a real need.*

Decision 2: LangGraph vs. Raw Python for State Management

The conference bot runs on raw Python with SQLite for state. This works for a single-user Telegram bot, but it won't scale to a multi-user, multi-archetype platform. The question is when to introduce LangGraph.

- **Pros of LangGraph:** Built-in checkpointing (conversations survive crashes), human-in-the-loop interrupts, time-travel debugging, persistent memory across sessions, graph visualization of agent behavior.
- **Cons of LangGraph:** Additional dependency, learning curve, slightly more complex deployment. Requires understanding of state machines and graph-based thinking.
- **Verdict:** Introduce LangGraph in Phase 2 (post-conference). The raw Python bot is fine for 50-100 conference users. LangGraph becomes essential when Navi supports multiple concurrent archetypes with persistent, resumable conversations.

Decision 3: Self-Hosted (Dify/Ollama) vs. API-Only

Running open-source infrastructure (Dify for workflows, Ollama for local inference) gives you control and reduces API costs, but adds ops burden. For a solo founder pre-PMF, the trade-off is clear:

- **Pre-PMF (now):** Stay API-only. OpenRouter handles inference. No self-hosting overhead. Focus 100% of energy on product and users, not infrastructure.

• **Post-PMF:** Introduce Dify for internal prototyping. Consider Ollama for cost reduction on high-volume, low-stakes tasks (like message classification or sentiment analysis).

• **Scale (100+ users):** Evaluate full self-hosting for cost control and latency. DigitalOcean droplet is already provisioned as a starting point.

2.4 Phased Implementation Roadmap

PHASE 0: CONFERENCE SHIP (Now → April 10, 2026)

Stack: Python + python-telegram-bot + OpenRouter + SQLite + Flask

New frameworks: None.

Focus: Ship the three remaining deliverables:

- Wire the Typeform webhook → 6-char code → SQLite → bot /start handoff
- Build the landing page (Next.js on Vercel, follows DESIGN.md)
- Design and print 50-100 QR cards with final domain URL

Success metric: Attendees can scan QR → complete questionnaire → start chatting with the bot within 2 minutes.

PHASE 1: ARCHETYPE PROTOTYPE (April 11 → May 15, 2026)

Stack adds: CrewAI

Goal: Build the first production archetype based on conference data.

Step 1: Analyze Conference Data

- Export all conversation logs, Typeform responses, and engagement metrics from SQLite.
- Identify which archetype had the highest engagement, message count, and follow-up conversion.
- Determine the winning archetype to build first (e.g., 'Barber Assistant' or 'Solo Attorney').

Step 2: Install and Configure CrewAI

- pip install crewai crewai-tools
- Scaffold the project: crewai create crew navi-archetype
- Define agents in agents.yaml: intake_agent, task_agent, knowledge_agent.
- Define tasks in tasks.yaml: onboarding, scheduling, reminders, Q&A.;
- Wire OpenRouter as the LLM backend (CrewAI supports OpenAI-compatible APIs).

Step 3: Build the Archetype Crew

- Port the existing bot's system prompts into CrewAI agent backstories.
- Add the context_library entries as knowledge for the knowledge_agent.
- Implement the morning message and follow-up flows as CrewAI tasks.
- Test the full flow: user onboarding → first conversation → follow-up sequence.

Step 4: Validate with Conference Users

- Invite the 10 most engaged conference participants to test the archetype.
- Collect qualitative feedback on personality, usefulness, and reliability.
- Iterate on agent backstories and task definitions based on feedback.

PHASE 2: PLATFORM FOUNDATION (May 16 → July 2026)

Stack adds: LangGraph + LangChain

Goal: Transform from a single-archetype prototype to a multi-archetype platform with durable state.

Step 1: Introduce LangGraph for State Management

- Define each agent's behavior as a LangGraph state machine (nodes = actions, edges = transitions).
- Enable checkpointing so conversations survive server restarts and crashes.
- Implement human-in-the-loop breaks for high-stakes actions (e.g., sending messages on user's behalf).
- Add persistent memory: short-term (conversation context) and long-term (user preferences, history).

Step 2: Add LangChain Tool Layer

- Wrap external APIs (Google Calendar, booking systems, payment processors) as LangChain tools.
- Build per-user RAG pipelines: document ingestion → embedding → retrieval for domain knowledge.
- Create a tool registry so new integrations can be added without modifying core agent code.
- Implement tool-use guardrails: rate limits, cost caps, permission checks.

Step 3: Build Second and Third Archetypes

- Use the CrewAI + LangGraph + LangChain stack to build two more archetypes.
- Each archetype shares the platform infrastructure but has unique agents, tools, and knowledge.
- Validate that the architecture supports archetype diversity without code duplication.

PHASE 3: SCALE & INTELLIGENCE (August 2026+)

Stack adds: Dify (internal) + AutoGen (selective) + Ollama (cost optimization)

Goal: Scale to 10+ archetypes, reduce costs, add advanced agent collaboration.

Step 1: Deploy Dify for Internal Prototyping

- Self-host Dify on the DigitalOcean droplet or dedicated server.
- Use Dify's visual workflow builder to rapidly prototype new archetype workflows.
- Once validated, port the workflow to the CrewAI/LangGraph production stack.

Step 2: Introduce AutoGen for Agent Collaboration

- Add AutoGen for archetypes that benefit from agent-to-agent review (e.g., content creation workflows).
- Use AutoGen's debate pattern: one agent drafts, another critiques, they converge on quality output.
- Keep AutoGen scoped to specific archetypes. Don't make it a platform-wide dependency.

Step 3: Cost Optimization with Ollama

- Run Ollama on the Mac Mini or DigitalOcean for high-volume, low-stakes tasks.
- Route message classification, sentiment analysis, and simple Q&A; to local models.
- Keep complex reasoning and personality-critical interactions on Claude/OpenRouter.
- Measure cost savings and latency trade-offs before expanding local inference.

Step 4: Add Evaluation and Observability

- Implement conversation quality metrics (engagement, task completion, user satisfaction).
- Set up MLflow for tracking agent experiments (prompt variations, model comparisons).
- Build automated regression tests: golden conversations that must produce acceptable outputs.
- Monitor for drift: agents should maintain personality and quality as the platform scales.

2.5 Architecture for Nimble Expansion

Navi's architecture must support two things that are in tension: a clear, decisive stack today and the flexibility to pivot based on what the market tells you. Here are the design principles that resolve that tension:

Principle 1: Abstraction Layers Over Direct Dependencies

Never let your business logic depend directly on a framework's API. Wrap every framework interaction behind a Navi-defined interface. If you replace CrewAI with AutoGen later, only the wrapper changes. Your archetype definitions, prompts, and user data stay untouched.

Principle 2: Config-Driven Archetypes

Each archetype should be defined entirely in configuration (YAML or JSON): its agents, their roles, tools, knowledge sources, and personality. The platform code reads this config and instantiates the right components. Adding a new archetype should never require writing new Python classes.

Principle 3: One Database, Multiple Stores

Keep SQLite (or Postgres when you scale) as the single source of truth for user data, conversations, and metrics. Use framework-specific stores (LangGraph checkpoints, vector DBs for RAG) as secondary caches that can be rebuilt from the primary database.

Principle 4: Feature Flags for Framework Adoption

Wrap every new framework behind a feature flag. Roll out LangGraph to 10% of users first. If something breaks, flip the flag and fall back to raw Python. This is how you move fast without breaking the product.

Principle 5: Measure Before Expanding

Never add a framework because it seems cool. Add it because you measured a specific gap: conversations aren't resuming after crashes (add LangGraph), agents need external tools (add LangChain), multiple agents need to coordinate (add CrewAI). The measurement is the permission to expand.

SECTION 3: WIRING OPEN-SOURCE FRAMEWORKS INTO CLAUDE COWORK

3.1 Integration Methods: How Cowork Consumes External Tools

Claude Cowork is not a traditional IDE. It is a sandboxed environment with file tools, a Linux shell, web access, and MCP server connections. You cannot just 'paste a codebase into a .md file' and expect Claude to use it as a framework. There are four distinct integration methods, each suited to different types of tools. Understanding which method to use is the key to making this work.

Method 1: Pip Install + Bash Execution

The most direct method. Cowork's sandboxed shell runs Ubuntu with Python 3.10+ and Node.js. You can install any pip package and execute Python scripts. This is how you use LangGraph, CrewAI, LangChain, AutoGen, and any other Python framework. The install persists for the duration of the session.

How it works:

- Claude runs: pip install crewai langgraph langchain --break-system-packages
- Claude writes a Python script to your workspace folder (e.g., agent_workflow.py)
- Claude executes: python agent_workflow.py
- Output is captured, analyzed, and presented to you in the chat

Best for: LangGraph, CrewAI, AutoGen, LangChain, PyTorch, Transformers, Optuna, MLflow

Limitation: Packages re-install each session. For heavy frameworks, include install commands in a setup script or skill.

Method 2: Skills (SKILL.md Files)

Skills are instruction files that teach Claude how to perform specific tasks. They live in your .claude/skills/ directory and are loaded when relevant. A skill does NOT contain the framework's source code. Instead, it contains instructions for Claude on how to install, configure, and use the framework. Think of it as a playbook, not a library.

How it works:

- Create .claude/skills/langgraph/SKILL.md with setup instructions, code templates, and usage patterns
- When you ask Claude to 'build a stateful agent workflow,' it reads the skill and follows the playbook
- The skill can include: install commands, boilerplate code, configuration templates, best practices
- Skills are version-controlled and persist across sessions (they live in your project directory)

Best for: Teaching Claude your preferred patterns for using any framework. Reusable across sessions.

This is NOT: Pasting the entire LangGraph source code into a markdown file. That would waste context and accomplish nothing.

Method 3: MCP Servers (Model Context Protocol)

MCP servers expose external tools that Claude can call natively. Instead of Claude writing a script and running it, an MCP server gives Claude direct function-call access to a tool's capabilities. This is the most seamless integration but requires a running server process.

How it works:

- You set up an MCP server (e.g., a CrewAI MCP server that exposes crew.kickoff() as a callable tool)
- Register the server in your Claude settings or .claude/settings.json
- Claude sees the tools as native functions and can call them directly in conversation
- No bash commands needed. Claude just calls the tool like it calls Read or Write

Best for: Dify (workflow triggers), LangGraph (state management), any external API you want Claude to call natively.

Limitation: Requires setting up and running the MCP server process. More involved than pip install.

Method 4: Claude Agent SDK (Subagents & Agent Teams)

The Agent SDK lets Claude spawn subprocesses that run their own agent loops. Each subagent can have different tools, different instructions, and even different models. This is the native way to build the multi-agent team described in Section 1, and it can be combined with external frameworks.

How it works:

- Install: pip install claude-agent-sdk (Python) or npm install @anthropic-ai/claude-agent-sdk (TypeScript)
- Define agents with specific tools, system prompts, and allowed actions
- Parent agent spawns subagents for focused subtasks, each in its own context window
- Subagents can use LangGraph, CrewAI, etc. internally while the parent orchestrates

Best for: Building the Chief/Researcher/Planner/Assistant team with framework-powered internals.

3.2 Framework-by-Framework Implementation Guide

For each framework, here is exactly how to make it available in Cowork, what file(s) to create, and where they go. No copy-pasting source code. No guessing.

CrewAI

Integration method: Pip Install + Skill + optional MCP

1. **Create the skill**: Write .claude/skills/crewai/SKILL.md containing: install command (pip install crewai crewai-tools), how to scaffold a crew (crewai create crew <name>), YAML templates for agents.yaml and tasks.yaml, OpenRouter wiring instructions, and example crew execution code.
2. **Install at session start**: The skill instructs Claude to run pip install crewai crewai-tools --break-system-packages when a CrewAI task is invoked. This takes about 15 seconds.
3. **Scaffold and run**: Claude follows the skill to create crew configs in your workspace, writes the Python execution script, and runs it. Output flows back to the chat.
4. **Optional MCP**: The crewai-claude MCP server (github.com/miudinho420/crewai-claude) exposes CrewAI operations as native tool calls. Set up if you want Claude to manage crews without writing scripts.

LangGraph

Integration method: Pip Install + Skill

1. **Create the skill**: Write .claude/skills/langgraph/SKILL.md containing: install command (pip install langgraph langchain-core), state machine patterns (nodes, edges, state schemas), checkpointing setup (MemorySaver for dev, SqliteSaver for prod), and templates for common agent patterns (ReAct, plan-and-execute).
2. **Define graph in Python**: Claude writes a graph definition file (e.g., navi_graph.py) that defines nodes as functions, edges as transitions, and state as a TypedDict. This is a regular Python file, not a config.
3. **Execute and iterate**: Claude runs the graph, inspects state at each checkpoint, and modifies the graph structure based on results. LangGraph's time-travel debugging lets Claude replay from any checkpoint.
4. **Persist for reuse**: Save the graph definition to your workspace. Future sessions can load and extend it without rebuilding from scratch.

LangChain

Integration method: Pip Install + Skill

1. **Create the skill**: Write .claude/skills/langchain/SKILL.md containing: install command (pip install langchain langchain-community langchain-openai), tool wrapping patterns, RAG pipeline templates

(document loader, text splitter, embeddings, vector store, retriever), and chain composition patterns.

2. Use selectively: Do not use LangChain as an orchestrator. Use it only for: wrapping external APIs as tools, building RAG pipelines, and document processing. Orchestration is handled by LangGraph or CrewAI.

3. Tool registry pattern: Create a tools/ directory in your workspace with one Python file per tool integration. Claude loads only the tools needed for the current task.

AutoGen

Integration method: Pip Install + Skill (deferred)

1. Create the skill: Write .claude/skills/autogen/SKILL.md containing: install command (pip install autogen-agentchat autogen-ext), agent patterns (AssistantAgent, UserProxyAgent), group chat setup, and the debate/critique pattern where two agents review each other's output.

2. When to activate: Only invoke this skill when a task genuinely benefits from agent-to-agent debate. Content review, code critique, and multi-perspective analysis are good triggers. Simple tasks should never use AutoGen.

3. Event-driven architecture: AutoGen v0.4 uses async message passing. The skill should include async boilerplate so Claude can write correct AutoGen code without fumbling the event loop.

Dify

Integration method: MCP Server (primary) or API calls via Bash

1. Self-host Dify: Run Dify on your Mac Mini or DigitalOcean droplet using Docker (docker compose up -d). Dify runs as a web application on port 80 by default.

2. Create workflows visually: Use Dify's web UI to build agent workflows. This is the whole point: visual, no-code workflow creation that Matt can do directly.

3. Connect to Cowork via API: Dify exposes a REST API. Create a skill (.claude/skills/dify/SKILL.md) that teaches Claude how to trigger Dify workflows via curl/Python requests. Include the API endpoint, auth token pattern, and common workflow IDs.

4. Optional MCP: Build a lightweight MCP server that wraps Dify's API, giving Claude native tool-call access to trigger workflows, check status, and retrieve results.

Ollama

Integration method: Bash (API calls to local server)

1. Install on Mac Mini: brew install ollama, then ollama pull llama3 (or whichever model). Ollama runs as a local server on port 11434.

2. Create the skill: Write .claude/skills/ollama/SKILL.md with: how to check if Ollama is running, how to query models via the REST API (curl http://localhost:11434/api/generate), and patterns for routing low-stakes tasks to local models.

3. Use as a cost router: Claude sends simple classification/sentiment tasks to Ollama's local endpoint and reserves Claude/OpenRouter for complex reasoning. The skill defines which task types route where.

3.3 File Structure: Where Everything Lives

Here is the exact directory structure for integrating frameworks into Cowork. This is not theoretical. This is where you put the files.

Path	Purpose
`your-project/`	
■■■ `.claude/`	
■ ■■■ `CLAUDE.md`	Your existing root instructions (don't touch)
■ ■■■ `settings.json`	MCP server registrations go here
■ ■■■ `skills/`	
■ ■■■ `crewai/`	
■ ■ ■■■ `SKILL.md`	CrewAI playbook: install, scaffold, run
■ ■■■ `langgraph/`	
■ ■ ■■■ `SKILL.md`	LangGraph playbook: graphs, state, checkpoints
■ ■■■ `langchain/`	
■ ■ ■■■ `SKILL.md`	LangChain playbook: tools, RAG, chains
■ ■■■ `autogen/`	
■ ■ ■■■ `SKILL.md`	AutoGen playbook: debate, critique, async
■ ■■■ `dify/`	
■ ■ ■■■ `SKILL.md`	Dify playbook: API calls, workflow triggers
■ ■■■ `ollama/`	
■ ■ ■■■ `SKILL.md`	Ollama playbook: local model routing
■ ■■■ `agent-team/`	
■ ■■■ `SKILL.md`	Team orchestration: Chief/Researcher/Planner/Assistant
■■■ `tools/`	Reusable Python tool wrappers (LangChain tools)
■■■ `workflows/`	Saved LangGraph graphs, CrewAI crew configs
■■■ `scripts/`	Standalone execution scripts

> *Key point: SKILL.md files contain instructions, not source code. They tell Claude what to install, how to configure it, and what patterns to follow. The actual framework code comes from pip install, not from pasting into a markdown file.*

3.4 Comparisons: Skills vs. MCP vs. Bash Scripts vs. Agent SDK

Each integration method has different trade-offs. Use the right one for the right framework.

Method	Persists Across Sessions?	Setup Effort	Execution Speed	Best For
Pip + Bash	No (reinstall each session)	Low	Fast	LangGraph, CrewAI, LangChain, AutoGen, PyTorch
Skill (SKILL.md)	Yes (in your project)	Low-Medium	N/A (instructions only)	Teaching Claude your patterns, templates, and preferences
MCP Server	Yes (registered in settings)	Medium-High	Fastest (native tool calls)	Dify, custom APIs, persistent services
Agent SDK	No (session-scoped)	Medium	Fast (parallel subagents)	Multi-agent team orchestration, subagent delegation
Skill + Pip (combined)	Skill persists, packages don't	Medium	Fast after install	The recommended pattern for most frameworks

The Recommended Pattern

For most frameworks, the best approach is: Skill + Pip Install. The skill file persists across sessions and contains the installation commands, configuration templates, and usage patterns. When the skill is invoked, Claude reads the instructions, installs the package, and executes. This gives you persistence (the skill survives session boundaries) and power (the full framework is available at runtime). You never paste framework source code into a file. You write a playbook that tells Claude how to use the framework.

What Goes in a SKILL.md (Template)

Every framework skill should follow this structure. This is the template Claude reads when the skill is invoked:

```
# [Framework Name] Skill

## Setup
- Install command: pip install [package] --break-system-packages
- Required env vars: OPENROUTER_API_KEY (or whatever the framework needs)
- Verify install: python -c 'import [package]; print([package].__version__)'

## Core Patterns
- [Pattern 1]: Description + code template
- [Pattern 2]: Description + code template

## Configuration Templates
- [Template 1]: YAML/JSON config for common use case
- [Template 2]: YAML/JSON config for another use case

## Common Pitfalls
- [Pitfall 1]: What goes wrong and how to fix it

## Example Execution
- Complete working example Claude can run immediately
```

3.5 Practical Quickstart: Your First Cowork + Framework Session

Here is a concrete, copy-this-and-go walkthrough for setting up your first framework-powered Cowork session using CrewAI (the recommended Phase 1 framework). After this, the same pattern applies to every other framework.

Step 1: Create the Skill File

- In your project's .claude/skills/ directory, create a folder called crewai/ with a SKILL.md file inside.
- The skill should contain: install instructions, a YAML template for defining agents, a YAML template for tasks, the Python code to execute a crew, and instructions for wiring OpenRouter as the LLM backend.
- This file is about 100-150 lines. It is instructions and templates, not framework source code.

Step 2: Register the Skill

- Skills in .claude/skills/ are automatically discovered by Claude Cowork when they match the task context.
- You can also invoke a skill explicitly by asking Claude to 'use the crewai skill' or by referencing the skill name.
- No additional registration is needed beyond placing the SKILL.md file in the right directory.

Step 3: Start a Session and Ask for a Crew

- Open Cowork and say: 'Build a CrewAI crew for a barber shop assistant with an intake agent, scheduling agent, and follow-up agent.'
- Claude reads the skill, runs pip install, scaffolds the crew in YAML, writes the execution script, and runs it.
- The entire process takes 2-5 minutes from request to working output.

Step 4: Iterate and Save

- Review the output. Ask Claude to modify agent backstories, add tools, change the task flow.
- Each iteration, Claude updates the YAML configs and reruns. No manual coding required.
- Save the final crew configs to your workspace folder. They are now reusable across sessions.

Step 5: Expand to Other Frameworks

- Repeat the same pattern for LangGraph, LangChain, etc.: create the SKILL.md, define templates, let Claude handle execution.
- Each framework gets its own skill file. They can be combined: a CrewAI crew that uses LangGraph for state management and LangChain for tool integrations. The skills tell Claude how each piece fits together.

The answer to 'do I just copy-paste them into a .md file?' is: No. You write a SKILL.md playbook that teaches Claude how to install, configure, and execute the framework. The framework itself is installed from pip at runtime. The skill is the brain. Pip is the muscle. Your workspace stores the output.

Bottom line: Ship the conference with what you have. Let the data tell you which archetype to build first. Then add frameworks in layers, one at a time, measuring each addition's impact. The stack above gives you a clear path from solo-bot to multi-archetype platform without painting yourself into a corner.

Prepared by Claude for Honeyheart (Matt) — March 30, 2026

navi. — Navigate your life.